

Movie Rental Database System

CS 157A — Final Project Report

Team Members:

Addison Schleigh — SID: 017595110

Haitham Assaf — SID: 017558515

Tyler Fabela — SID: 016347201

Date Submitted: May 11, 2025

1. Introduction

1.1 Project Overview

This project is a Movie Rental Database System we built for CS 157A. The application lets store staff manage a movie catalog, register customers, process rentals and returns, and record customer ratings. It is relevant because video rental stores need to coordinate data across many domains (inventory, customers, active rentals, returns, and feedback), and a properly normalized relational database is the right tool for the job.

We built the system as a three-tier application: a Java Swing desktop GUI for the user, Java DAO classes that send SQL through JDBC, and a MySQL database normalized to BCNF. Building it this way means each layer can change without breaking the others.

1.2 Chosen Topic

We chose the Movie Rental System project from the list of suggested topics. Use case: a clerk at a video rental store needs to look up a movie, check whether copies are available, rent it to a customer, and later process the return with any late fees. They also need to register new customers and record what customers think of the movies they watched. Our system supports each of these workflows from a single application window.

2. Objectives

2.1 Primary Goals

- Develop a fully functional movie rental management system.
- Design a relational schema with at least four tables, all in BCNF.
- Populate every table with at least fifteen rows of representative sample data, with no NULL values.
- Build a Java application that can run SELECT, INSERT, UPDATE, and DELETE on every table through JDBC.
- Provide a graphical user interface so staff never have to type raw SQL.

2.2 Key Features

- Full CRUD (Create, Read, Update, Delete) on all five tables.
- Search by movie title (LIKE pattern) and by customer name.
- Rent-a-movie workflow: select a movie, pick a customer from a dropdown, the system creates the rental and decrements available copies.
- Process-return workflow: select an active rental, the system auto-calculates a late fee, creates a Return record, marks the rental Returned, and increments available copies.
- Filter active rentals only.

- Compute average rating score per movie.

3. System Architecture

3.1 Three-Tier Architecture

The system is split into three layers, each in its own Java package:

- Presentation Layer (ui package): A Swing GUI with a JFrame and JTabbedPane. Each of the five tabs corresponds to one table and shows a JTable plus buttons for Add, Edit, Delete, and Search.
- Logic Layer (dao and model packages): Plain Java model classes mirror table rows. DAO classes hold all the SQL. The GUI never builds a SQL string directly — it asks a DAO method, which runs the query through JDBC.
- Database Layer (db package and the MySQL server): The DatabaseConnection class is the only place that opens a JDBC connection. The schema and sample data live in the movie_rental MySQL database.

3.2 Technology Stack

- Database: MySQL Server 9.7
- Database API: JDBC with the MySQL Connector/J driver (mysql-connector-j-9.7.0.jar)
- Programming Language: Java (JDK 17)
- UI Toolkit: Java Swing (JFrame, JTabbedPane, JTable, JOptionPane, JDialog)
- Development Environment: IntelliJ IDEA / VS Code
- Version Control: Git and GitHub

3.3 Database Connection

All database access starts at the DatabaseConnection class. It uses DriverManager.getConnection with the JDBC URL jdbc:mysql://localhost:3306/movie_rental, plus a username and password. The MySQL Connector/J jar must be on the classpath at compile time and run time.

Setup steps to get the connection working:

- Install MySQL Server locally and run our schema.sql and data.sql scripts to create the movie_rental database and load sample data.
- Download mysql-connector-j-9.7.0.jar and place it in the lib folder.
- Edit DatabaseConnection.java to set the local MySQL password.
- Compile with the connector on the classpath, then run.

Sample connection code:

```
private static final String URL      = "jdbc:mysql://localhost:3306/movie_rental";
private static final String USER    = "root";
private static final String PASSWORD = "****";

public static Connection getConnection() throws SQLException {
    return DriverManager.getConnection(URL, USER, PASSWORD);
}
```

4. Database Design

4.1 Schema Diagram

The schema has five tables. Primary keys are underlined in the relations below, and foreign keys are noted in the constraints table in section 4.4. The full E/R diagram is included as a separate file

(ER_Diagram.pdf) in the submission package and shows entities as rectangles, the weak entity Returns as a double rectangle, relationships as diamonds, and cardinalities (1:M, N:1, 1:1) on each connection.

4.2 BCNF Relations

All five relations are in BCNF. The relations are:

- Movies(MovieID, Title, Genre, Director, ReleaseYear, MPAARating, CopiesAvailable, RentalPrice)
- Customers(CustomerID, FirstName, LastName, Email, Phone, Address, JoinDate)
- Rentals(RentalID, CustomerID, MovieID, RentalDate, DueDate, Status)
- Returns(ReturnID, RentalID, ReturnDate, LateFee)
- Ratings(RatingID, CustomerID, MovieID, Score, ReviewDate)

4.3 Entity Descriptions

- Movies: stores the catalog. Each row is one title with its genre, director, year, MPAA rating, number of copies in stock, and rental price.
- Customers: stores account info for each registered renter — name, email, phone, address, and the date they joined.
- Rentals: stores active and past rental transactions. Each row links one customer to one movie with rental and due dates plus a status (Active / Returned / Overdue).
- Returns: stores return records. Each row is tied to exactly one Rental and records the return date and any late fee charged.
- Ratings: stores customer ratings. Each row is one customer's score (1-5) for one movie on a given review date.

4.4 Attributes and Constraints

Table	Attribute	Constraints
Movies	MovieID	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
	Title, Genre, Director, MPAARating	VARCHAR, NOT NULL
	ReleaseYear, CopiesAvailable	INT, NOT NULL
	RentalPrice	DECIMAL(5,2), NOT NULL
Customers	CustomerID	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
	Email	VARCHAR, NOT NULL, UNIQUE
	FirstName, LastName, Phone, Address	VARCHAR, NOT NULL
	JoinDate	DATE, NOT NULL
Rentals	RentalID	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
	CustomerID	FOREIGN KEY → Customers, NOT NULL
	MovieID	FOREIGN KEY → Movies, NOT NULL
	RentalDate, DueDate	DATE, NOT NULL

	Status	VARCHAR, NOT NULL
Returns	ReturnID	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
	RentalID	FOREIGN KEY → Rentals, NOT NULL, UNIQUE
	ReturnDate	DATE, NOT NULL
	LateFee	DECIMAL(5,2), NOT NULL
Ratings	RatingID	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
	CustomerID	FOREIGN KEY → Customers, NOT NULL
	MovieID	FOREIGN KEY → Movies, NOT NULL
	Score	INT, NOT NULL (validated 1-5 in app)
	ReviewDate	DATE, NOT NULL

5. Functional Requirements

5.1 User Roles and Access

The system has one user role: store staff (clerks and managers). Staff launch the desktop GUI on a workstation in the store and use it for all daily operations. The application is for internal store use; end customers do not interact with it directly. There is no login screen in this version because the assumption is that access is controlled at the OS level on the staff workstation.

5.2 Feature Descriptions

Adding new records:

- Add Movie: clicks Add Movie on the Movies tab, fills in title/genre/director/year/MPAA/copies/price, clicks OK. The system runs an INSERT into Movies.
- Register Customer: clicks Add Customer on the Customers tab, fills in name/email/phone/address/join date, clicks OK. INSERT into Customers.
- Rent This Movie: selects a movie, clicks Rent This Movie, picks a customer from the dropdown. INSERT into Rentals + UPDATE Movies (CopiesAvailable - 1).
- Process Return: selects an active rental, clicks Process Return. The late fee field is pre-filled based on today vs the due date. INSERT into Returns + UPDATE Rentals.Status + UPDATE Movies.CopiesAvailable.
- Add Rating: clicks Add Rating on the Ratings tab, fills in CustomerID, MovieID, score, and date. INSERT into Ratings.

Viewing existing data:

- Each tab shows a JTable that lists every row in the corresponding database table. Clicking Show All re-runs SELECT * to refresh the view.

Updating records:

- Select any row in any tab, click Edit. The form opens pre-filled with current values. Changes are saved with UPDATE.

Deleting records:

- Select any row, click Delete. A confirmation dialog appears; on Yes, DELETE FROM table runs.

Additional functionality:

- Search Movies by title (LIKE %keyword% pattern).

- Search Customers by first or last name.
- Filter Rentals to show only Active ones.
- Filter Ratings by MovieID.
- Compute the average score for a given MovieID using AVG(Score).

6. Non-Functional Requirements

- Performance: Indexes are automatically created on all primary keys and on the UNIQUE Email column. Frequent filter operations on Rentals and Ratings run on indexed columns.
- Scalability: The schema and three-tier separation make it straightforward to add more tables or expand existing ones without rewriting existing code. The MySQL backend can handle millions of rows for our use case.
- Security: Database credentials live in a single class so they can be rotated easily. All queries use PreparedStatement with parameter binding, which protects against SQL injection. Forms validate basic input (numeric fields, date format, score 1-5) before sending the SQL.
- Accessibility: The GUI uses standard Swing components which respect OS-level font scaling. Buttons have descriptive labels, common actions are reachable in two clicks, and destructive actions ask for confirmation.

7. Implementation Details

7.1 Code Structure

The source code is organized by responsibility:

```
MovieRentalSystem/
├── sql/
│   ├── schema.sql      (table definitions, BCNF, NOT NULL, FK)
│   └── data.sql        (sample data, 15+ rows per table)
├── src/
│   ├── db/
│   │   └── DatabaseConnection.java
│   ├── model/          (Movie, Customer, Rental, Return, Rating)
│   ├── dao/            (one DAO per model with full CRUD)
│   └── ui/             (one Swing panel per table + main app)
├── lib/                (JDBC driver, not committed to Git)
└── README.md
```

Naming conventions: Java classes use PascalCase. SQL tables use PascalCase too (Movies, Customers). Methods like getAllMovies, insertMovie, updateMovie make their purpose obvious.

7.2 Important Code Snippets

Database connection (DatabaseConnection.java):

```
public static Connection getConnection() throws SQLException {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
    } catch (ClassNotFoundException e) {
        throw new SQLException("MySQL JDBC Driver not found.", e);
    }
    return DriverManager.getConnection(URL, USER, PASSWORD);
}
```

SELECT example with a parameterized search (MovieDAO):

```
public List<Movie> searchByTitle(String keyword) throws SQLException {
    String sql = "SELECT * FROM Movies WHERE Title LIKE ? ORDER BY Title";
    List<Movie> movies = new ArrayList<>();
```

```

    try (Connection conn = DatabaseConnection.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString(1, "%" + keyword + "%");
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) movies.add(buildMovie(rs));
        }
    }
    return movies;
}

```

INSERT example (MovieDAO):

```

public boolean insertMovie(Movie m) throws SQLException {
    String sql = "INSERT INTO Movies (Title, Genre, Director, ReleaseYear, " +
        "MPAARating, CopiesAvailable, RentalPrice) " +
        "VALUES (?, ?, ?, ?, ?, ?, ?)";
    try (Connection conn = DatabaseConnection.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString(1, m.getTitle());
        // ... bind remaining 6 parameters ...
        return pstmt.executeUpdate() > 0;
    }
}

```

7.3 Error Handling

Database errors are caught at the GUI layer. Every DAO method declares throws SQLException, and the calling Swing panel wraps the call in a try-catch block. When an exception occurs, the panel shows a JOptionPane error dialog so the user knows what went wrong without the program crashing. Example pattern from MoviePanel:

```

try {
    movies = dao.getAllMovies();
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(this,
        "Failed to load movies: " + ex.getMessage(),
        "Error", JOptionPane.ERROR_MESSAGE);
}

```

On startup, MovieRentalApp opens a test connection. If the database is unreachable or credentials are wrong, a clear error dialog appears and the app exits gracefully instead of crashing later. Forms also validate basic input (numeric fields parsed via Integer.parseInt with NumberFormatException catches, dates parsed via Date.valueOf with IllegalArgumentException catches).

8. Testing and Validation

8.1 Test Cases

We tested every feature manually through the GUI. Test cases:

#	Test Case	Expected	Result
1	Open the app while MySQL is running	Window opens, all 5 tabs load data	Pass
2	Open the app while MySQL is stopped	Error dialog, app exits cleanly	Pass
3	Add a movie with all fields filled in	New row appears in Movies table	Pass

4	Edit a movie's price	Updated price shows after refresh	Pass
5	Delete a movie with no rentals	Row removed from table	Pass
6	Delete a movie referenced by a rental	FK constraint error in dialog	Pass
7	Search Movies for keyword "Star"	Only matching titles shown	Pass
8	Register a customer	New row in Customers	Pass
9	Register a customer with duplicate email	UNIQUE error dialog	Pass
10	Rent a movie with copies > 0	Rental created, copies decrement	Pass
11	Rent a movie with copies = 0	Out of stock warning	Pass
12	Process return on time	Late fee = \$0.00, copies + 1	Pass
13	Process return 3 days late	Late fee pre-filled at \$4.50	Pass
14	Process return on already-Returned rental	Warning dialog, no changes	Pass
15	Submit a rating with score 6	Validation error, score must be 1-5	Pass
16	Show Average for a movie with ratings	AVG(Score) shown in dialog	Pass

8.2 Sample Data

data.sql loads the following sample data, with no NULL values anywhere:

- Movies: 40 rows (mix of action, drama, sci-fi, animation, comedy, etc.)
- Customers: 20 rows (San Jose area addresses)
- Rentals: 25 rows (mix of Active and Returned status)
- Returns: 15 rows (one per Returned rental, mix of on-time and late)
- Ratings: 28 rows (scores 3-5, spread across many movies)

8.3 Test Results

All 16 test cases pass. CRUD works on every table. Foreign-key violations and UNIQUE-constraint violations are caught and displayed as user-friendly dialogs instead of crashing the program. Auto-decrement and auto-increment of CopiesAvailable behave correctly during rent and return workflows. Late-fee calculation correctly produces \$0.00 for on-time returns and \$1.50 per day past due otherwise.

9. Challenges and Solutions

- Challenge: Initial data.sql failed because some Rentals referenced MovieID 40 but the Movies INSERT only had 38 rows. Solution: added the missing two movies so foreign-key constraints would not reject the rentals on load.

- Challenge: Each team member needed a different MySQL password locally, which made DatabaseConnection.java a constant source of merge conflicts. Solution: agreed on a shared password for everyone's local install so the file does not need editing on each pull.
- Challenge: When testing delete operations, deleting a movie that had a rental threw a confusing exception. Solution: caught SQLException at the GUI layer and showed a clear error dialog explaining the constraint violation, instead of letting the exception bubble up.
- Challenge: Raw CRUD on the Rentals tab forced the user to memorize CustomerID and MovieID. Solution: added "Rent This Movie" on the Movies tab (with a customer dropdown) and "Process Return" on the Rentals tab (with auto-calculated late fee). Both buttons run multiple SQL statements behind the scenes so the user only sees movie titles and customer names, not IDs.

10. Future Enhancements

- Replace Swing with a JSP/Servlet web front end so staff could access the system from any browser. Because of the three-tier separation, only the presentation layer would change.
- Add a stored procedure or trigger that auto-computes the late fee on Return insert, instead of doing the math in Java.
- Add database triggers that decrement and increment CopiesAvailable automatically, so the application code does not have to remember to do it.
- Add role-based authentication: clerks can rent and return; managers can also delete and edit catalog entries.
- Add reporting queries: top-rented movies, highest-rated movies, total late-fee revenue per month.
- Replace the MovieID dropdown in Add Rating with a dropdown of titles, the same way Rent This Movie now uses a customer dropdown.

11. Conclusion

11.1 Summary

We built a working Movie Rental Database System that pulls together every major topic from CS 157A. We started with an entity-relationship diagram, mapped it to a relational schema, verified that every table is in BCNF, then built a Java application that talks to the database through JDBC inside a three-tier architecture. The end result supports all four core SQL operations across five normalized tables, plus higher-level workflows (rent a movie, process a return) that combine multiple SQL statements behind a clean GUI.

11.2 Reflections

What went well: splitting the code into ui, dao, model, and db packages made it easy to divide the work across the team. The DAO pattern in particular kept the SQL out of the GUI code so individual panels stayed simple. PreparedStatement use across the board meant we never had to debug SQL injection issues.

What could be improved: in the next project we would set up Git and a shared development environment from day one instead of bouncing zip files around at the start. We would also write at least a few JUnit tests for the DAO methods so we could refactor with more confidence later, instead of relying entirely on manual GUI testing.

Appendix A. Database Schema SQL

Full schema.sql used to create the database (also included as a separate file in the submission):

```
DROP DATABASE IF EXISTS movie_rental;
CREATE DATABASE movie_rental;
```



```

USE movie_rental;

CREATE TABLE Movies (
    MovieID          INT AUTO_INCREMENT PRIMARY KEY,
    Title            VARCHAR(150)   NOT NULL,
    Genre            VARCHAR(50)    NOT NULL,
    Director         VARCHAR(100)   NOT NULL,
    ReleaseYear      INT            NOT NULL,
    MPAARating       VARCHAR(10)    NOT NULL,
    CopiesAvailable  INT            NOT NULL,
    RentalPrice      DECIMAL(5,2)   NOT NULL
);

CREATE TABLE Customers (
    CustomerID INT AUTO_INCREMENT PRIMARY KEY,
    FirstName  VARCHAR(50)   NOT NULL,
    LastName   VARCHAR(50)   NOT NULL,
    Email      VARCHAR(100)  NOT NULL UNIQUE,
    Phone      VARCHAR(20)   NOT NULL,
    Address    VARCHAR(200)  NOT NULL,
    JoinDate   DATE          NOT NULL
);

CREATE TABLE Rentals (
    RentalID   INT AUTO_INCREMENT PRIMARY KEY,
    CustomerID INT            NOT NULL,
    MovieID    INT            NOT NULL,
    RentalDate DATE          NOT NULL,
    DueDate    DATE          NOT NULL,
    Status     VARCHAR(20)   NOT NULL,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),
    FOREIGN KEY (MovieID)    REFERENCES Movies(MovieID)
);

CREATE TABLE Returns (
    ReturnID   INT AUTO_INCREMENT PRIMARY KEY,
    RentalID   INT            NOT NULL UNIQUE,
    ReturnDate DATE          NOT NULL,
    LateFee    DECIMAL(5,2)  NOT NULL,
    FOREIGN KEY (RentalID) REFERENCES Rentals(RentalID)
);

CREATE TABLE Ratings (
    RatingID   INT AUTO_INCREMENT PRIMARY KEY,
    CustomerID INT            NOT NULL,
    MovieID    INT            NOT NULL,
    Score      INT            NOT NULL,
    ReviewDate DATE          NOT NULL,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),
    FOREIGN KEY (MovieID)    REFERENCES Movies(MovieID)
);

```

Appendix B. Sample Queries

A few of the SQL queries our application runs:

SELECT — list all rentals for a given customer:

```
SELECT r.RentalID, m.Title, r.RentalDate, r.DueDate, r.Status
FROM Rentals r
JOIN Movies m ON r.MovieID = m.MovieID
WHERE r.CustomerID = ?;
```

SELECT — average rating per movie:

```
SELECT AVG(Score) AS AvgScore
FROM Ratings
WHERE MovieID = ?;
```

INSERT — create a new rental:

```
INSERT INTO Rentals (CustomerID, MovieID, RentalDate, DueDate, Status)
VALUES (?, ?, ?, ?, 'Active');
```

UPDATE — mark a rental as returned:

```
UPDATE Rentals SET Status = 'Returned' WHERE RentalID = ?;
```

DELETE — remove a customer:

```
DELETE FROM Customers WHERE CustomerID = ?;
```